

Würzburger Institut für Verkehrswissenschaften
(Institute for Traffic Sciences)



Robert-Bosch-Straße 4, 97209 Veitshöchheim
Kontakt: silab@wiw.de

SILAB[®] **Referenz Fuß-** **gängersimulation** **MOV**

Version 7.1

Stand: September 2022

INHALT

INHALT 2

1	ÜBERBLICK.....	3
2	KONFIGURATION.....	3
2.1	Konfiguration der DPUs	3
2.1.1	Grundlagen	3
2.1.2	Einbinden der Fußgängersimulation	4
2.1.3	Konfigurationsparameter	5
2.2	Konfiguration von MOV („MOP.inc“)	5
3	DEFINITION VON FUßGÄNGERN.....	5
3.1	Verwendung von Fußgängern in Area2 und Anwendungsbeispiele	5
3.1.1	Einbinden von autonom agierenden Fußgängern	5
3.1.2	Anwendung von Fußgängergruppen	6
3.1.3	Wichtige Hinweise und Einschränkungen	8
3.2	Scripting der MOV-Detailkontrolle	10
3.2.1	Fußgängergruppen	10
3.2.2	Skriptsprache	10
3.2.3	Einbindung in SILABAEEdit	22
3.2.4	Beispiel	23
4	REFERENZ	27
4.1	Referenz der Standard-Fußgänger	27
4.2	Referenz der Sonder-Fußgänger	31

1 ÜBERBLICK

Das SILAB-Modul MOV ermöglicht die Simulation von Fußgängern innerhalb von Stadtszenarien, die mit der Datenbasis SCNX realisiert wurden.

Mit MOV ist es sehr einfach, autonome Fußgänger zu simulieren. Dazu müssen mit dem Editor SILABAEEdit Fußgängerbereiche definiert werden, auf denen die Fußgänger dann umhergehen. Die virtuellen Personen wählen eigenständig aus einer Menge vordefinierter Ziele und weichen auf dem Weg dorthin selbständig Hindernissen, autonomen Fahrzeugen, die mit dem Modul TRFX realisiert werden, und dem Testfahrer aus. Auch Fußgängerampeln und –überwege werden regelgemäß benutzt. Zur Darstellung der Fußgänger werden 3D-Modelle im SIG-Dateiformat benutzt, von denen derzeit 35 im Lieferumfang von SILAB enthalten sind. Somit werden diese nahtlos in die Darstellung der Datenbasis durch SCNXSGE und TRFSGE integriert. Mit einer eigens entwickelten Animations-Engine werden Geh- und Laufbewegungen bei beliebigen Geschwindigkeiten zwischen 0 und ca. 14 km/h und andere Aktionen, die beispielsweise beim Warten an Ampeln auftreten, realitätsnah dargestellt. Des Weiteren besteht die Möglichkeit, einzelne Personen genauer zu steuern, um kritische Verkehrssituationen gezielt herstellen zu können. Beispielsweise kann mit SILABAEEdit ein Fußgänger so konfiguriert werden, dass er bei Annäherung des Testfahrers die Straße überquert und diesen so zum Handeln zwingt, um eine Kollision zu vermeiden.

Von MOV nicht unterstützt wird die Simulation von Fußgängern außerhalb von Area2-Typen¹, also auf Streckenabschnitten (Course-Instanzen). In der Praxis stellt dies keine große Einschränkung dar, da wegen dem gleich bleibenden Straßenquerschnitt in Course-Instanzen ohnehin nur lange, gerade Gehwege möglich wären. Außerdem werden Course-Instanzen meist dazu benutzt, Überland-Strecken zu realisieren, auf denen wenig Fußgängerverkehr zu erwarten ist. Sollten doch einmal für ein spezielles Szenario Fußgänger außerhalb von Ortschaften benötigt werden, ist es meist leicht möglich, den entsprechenden Streckenabschnitt als Area2 in SILABAEEdit zu konstruieren.

2 KONFIGURATION

2.1 Konfiguration der DPUs

2.1.1 Grundlagen

Die Fußgängersimulation MOV besteht aus den DPUs DPUMOV, DPUMOVClient und DPUMOPSGE.

Dabei wird innerhalb der **DPUMOV** die eigentliche Simulation der Fußgänger durchgeführt, d.h. ihre Bewegungen und Animationen werden geplant. Außerdem dient sie als Netzwerkserver für die DPUMOVClient-Instanzen. Sie wird üblicherweise auf dem Hauptrechner des Simulators gestartet. Wenn dieser aber bereits durch andere Aufgaben, wie z.B. eine aufwändige Verkehrssimulation oder Datenaufzeichnung, stark ausgelastet ist, empfiehlt es sich, die Bewegungsplanungs-Algorithmen in einem se-

¹ s. Dokumentation SILABAEEdit

paraten Thread (auf einem separaten Prozessorkern) rechnen zu lassen. Dieses Verhalten kann durch den DPU-Parameter *Threaded = 1* aktiviert werden.

Die **DPUMOVClient** bewegt Fußgänger synchron zur Simulation im entsprechenden Server, der in der DPUMOV implementiert ist. Die DPUs sind über ein lokales Netzwerk verbunden. Diese DPU wird auf den Grafikrechnern instanziiert. Falls DPUMOV nicht auf demselben Rechner wie DPUTRFX läuft, muss auch auf dem Rechner, auf dem der Verkehr simuliert wird, die DPUMOVClient gestartet werden.

Die **DPUMOPSGE** stellt Fußgänger grafisch in der Simulation dar und animiert sie. Die Simulationsdaten werden von der DPUMOV bzw. DPUMOVClient bezogen. Sie wird üblicherweise auf den Grafikrechnern gestartet.

Die einzelnen DPUs der Fußgängersimulation sind von folgenden anderen DPUs abhängig:

DPU	Benötigt
DPUMOV	DPUSCNX erforderlich
DPUMOVClient	DPUTRFX oder DPUTRFClient empfohlen
DPUMOPSGE	DPUSCNX + Netzwerkverbindung zu DPUMOV [DPUMOV oder DPUMOVClient] und DPUSGEWorld

Bei den Abhängigkeiten ist darauf zu achten, dass die abhängige DPU einen höheren *Index* erhält als die benötigten DPUs.

Die DPUMOV kann auf dem Hauptrechner der Simulation gestartet werden. Falls es sich dabei mindestens um einen Dual-Core Prozessor handelt, wird empfohlen, die Bewegungsplanung der DPUMOV in einem separaten Thread durchführen zu lassen (s.u.). Auf den Grafikrechnern des Simulators läuft jeweils DPUMOVClient und DPUMOPSGE.

Bei Einzelplatzsetups wird nur DPUMOV und DPUMOPSGE benötigt.

2.1.2 Einbinden der Fußgängersimulation

Die DPUs zur Fußgängersimulation sind Teil der SILAB-Standardkonfiguration.

Der DPU-Pool `Full` aus der Datei

```
SILAB\lib\configurations\system\DPUBase_SGE.inc
```

enthält bereits die nötigen Definitionen. Sie können ihn mit der Anweisung

```
include system\DPUBase_SGE.inc
```

in Ihre Versuchskonfiguration integrieren.

Die Definition der MOV-DPUs als Teil des Pool `Full` liegt in der Datei

```
SILAB\lib\configurations\system\DPUMOV_SGE.inc.
```

Hinweis:

Bitte führen Sie keine Änderungen an dieser Datei durch, wenn Sie Parameter der DPUs anpassen möchten. Beide oben genannten Dateien werden bei einem Update der SILAB-Version überschrieben.

Simulatorspezifische Anpassungen (z.B. der Parameter „NumPersons“ und „Threaded“) können Sie in der Datei

```
SILAB\lib\configurations\system\SYSMOVParams.inc
```

eintragen, die nicht von einem Update betroffen ist. Möglicherweise müssen Sie diese Datei zunächst anlegen.

Ein Beispiel für mögliche Inhalte der Datei `SYSMOVParams.inc` finden Sie im nächsten Abschnitt.

2.1.3 Konfigurationsparameter

Für die DPUMOV und ggf. die DPUMOVClient sollte vor allem der Parameter **NumPersons** gesetzt werden. Dieser legt die Maximalanzahl von Fußgängern in einem einzigen Szenario (Area2-Instanz) fest und damit auch die Rechenlast, die maximal durch die Fußgängersimulation auf den Rechnern erzeugt wird. Der Wert NumPersons hat Vorrang vor der Einstellung beim Export des Szenarios in SILABAEEdit (siehe Abbildung 1): Selbst wenn ein Szenario eine höhere Anzahl von Fußgängern anfordert, werden diese nicht alle simuliert. Üblicherweise können in Einzelplatzkonfigurationen maximal 60 Fußgänger in Echtzeit simuliert werden. In einem Simulator mit Rechnernetz werden etwa 100 Fußgänger unterstützt. Die Standardkonfiguration

`SILAB\lib\configurations\system\DPUMOV_SGE.inc`

legt die Maximalzahl unterstützter Fußgänger auf 100 fest.

Beispiel:

Um die Fußgängersimulation auf einen eigenen Thread zu verlagern und die maximale Anzahl von Fußgängern zu erhöhen, legen Sie die Datei

`SILAB\lib\configurations\system\SYSMOVParams.inc`

mit folgendem Inhalt an:

```
MOV.Threaded = 1;
MOV.NumPersons = 150;
MOVClient.NumPersons = 150;
```

Weitere Parameter der DPU-Konfiguration werden in den Dokumenten „DPUMOV“, „DPUMOVClient“ und „DPUMOPSGE“ beschrieben.

2.2 Konfiguration von MOV („MOP.inc“)

Die Fußgängersimulation MOV kann weitergehend konfiguriert werden, um neue Fußgängermodelle und –Animationen hinzuzufügen. Die Konfigurationsdatei haben wir bei der SILAB-Installation bereits für Sie eingerichtet. Daher muss bei den meisten Simulationsfahrten lediglich die Standard-Konfiguration eingebunden werden, die im Lieferumfang von SILAB enthalten ist. Dazu muss in der Versuchskonfiguration (Haupt-CFG-Datei) auf oberster Ebene folgende Zeile hinzugefügt werden:

```
include MOV\MOP.inc
```

Näheres zur Bearbeitung von Versuchskonfigurationen findet sich im Dokument „Referenz SILAB-Konfiguration und Bedienung“.

3 DEFINITION VON FUßGÄNGERN

3.1 Verwendung von Fußgängern in Area2 und Anwendungsbeispiele

3.1.1 Einbinden von autonom agierenden Fußgängern

Wie bereits erwähnt, kann die Simulation von Fußgängern leicht über SILABAEEdit in vorhandene Situationen integriert werden. Hierzu verwenden Sie das Werkzeug Fußgänger-Bereiche  im Layer Verkehr  des Editors SILABAEEdit.

Fügen Sie ebenfalls mindestens zwei Aufsetzpunkte  für Fußgänger innerhalb der Fußgänger-Bereiche hinzu. Die Bedienung dieser Werkzeuge wird im Dokument SILABAEEdit genauer erklärt.

Stellen Sie dann im Exportdialog so ein, dass der Verkehrsknoten als Modul mit TRF und MOP exportiert wird und geben Sie die Anzahl zu simulierender Fußgänger ein.

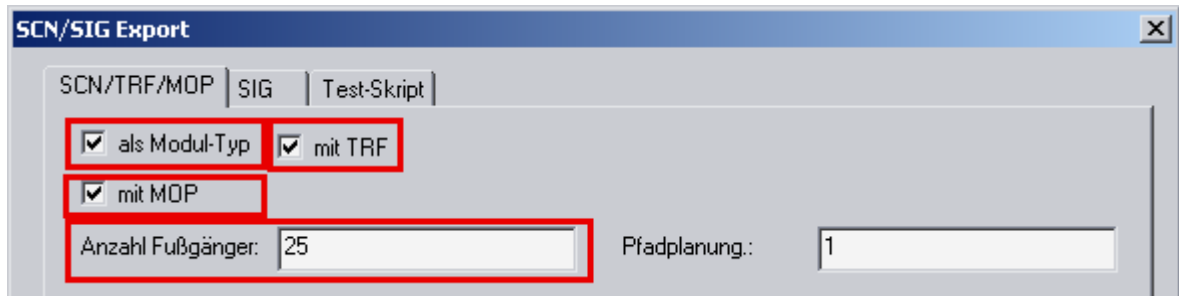


Abbildung 1 Einstellungen für den Export eines Area2

3.1.2 Anwendung von Fußgängergruppen

Mit Hilfe von Fußgängergruppen können die Fußgänger innerhalb eines Verkehrsknotens in mehrere Gruppen aufgeteilt werden. Dazu werden die Aufsetzpunkte unterschiedlichen Gruppen zugeordnet (siehe Abbildung 2).

Dies ermöglicht die genauere Steuerung der Fußgänger und den gezielten Einsatz von Fußgängern als Hindernis für den Testfahrer.

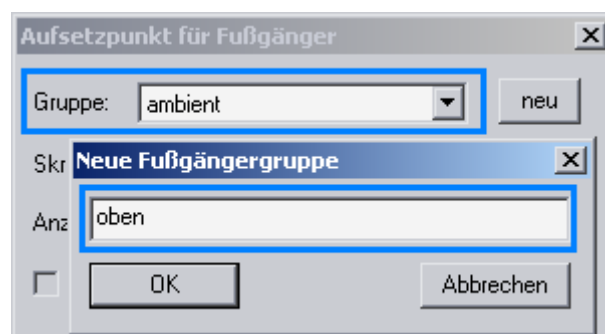


Abbildung 2 Auswahl und Erstellung von Fußgängergruppen

Aufbau nicht zusammenhängender Fußgängerbereiche

In manchen Fällen ist es nicht erwünscht, Fußgängerbereiche zwischen allen Aufsetzpunkten eines Verkehrsknotens einzurichten. Ein Beispiel ist in der folgenden Abbildung 3 zu sehen: eine breite Innerortsstraße ohne Überwege.

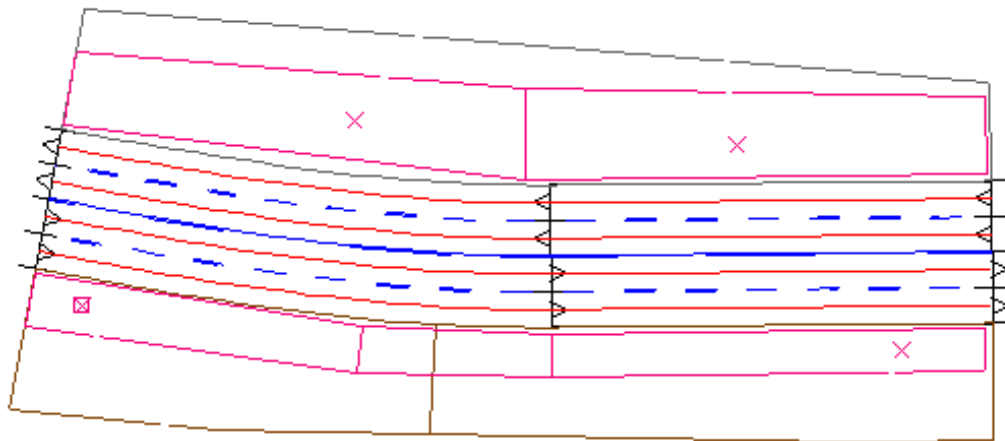


Abbildung 3 Aufbau nicht zusammenhängender Fußgängerbereiche

Da auf beiden Seiten Gehwege vorhanden sind, sollen sich dennoch links und rechts der Straße Fußgänger befinden.

Seit SILAB 4.0 kann in diesem Fall dennoch die Standard-Fußgängergruppe „ambient“ verwendet werden. (In vorigen Versionen war es notwendig, separate Fußgängergruppen für die linke und rechte Straßenseite zu erstellen.)

Beachten Sie bitte, dass in jedem Fußgängerbereich mindestens zwei Aufsetzpunkte vorhanden sein müssen, damit die Fußgänger zwischen diesen hin- und hergehen können.

Die im Verkehrsknoten insgesamt vorhandenen Fußgänger werden gleichmäßig auf die Aufsetzpunkte verteilt, d.h. je mehr Aufsetzpunkte auf einer Straßenseite vorhanden sind, desto mehr Fußgänger befinden sich während der Simulation auf dieser Seite.

Spezial-Einsatz von Fußgängern

Über die Auswahlbox „Skript“ der Aufsetzpunkt-Konfiguration kann ein besonderes Verhalten für alle Fußgänger, die zu dieser Gruppe gehören, definiert werden. Hiermit kann das Verhalten dieser Fußgänger drastisch verändert werden. Im Lieferumfang von SILAB befindet sich z.B. ein Skript, das dazu führt, dass die Fußgänger zunächst an einem Startpunkt der Fußgängergruppe stehen bleiben und erst dann losgehen, wenn sich der Testfahrer bis auf eine gewisse Distanz angenähert hat. Dieser Abstand kann als Parameter des Skripts im Aufsetzpunkt-Dialog eingestellt werden (siehe Abbildung 4). Üblicherweise wird nur ein einziger Fußgänger in diese Gruppe aufgenommen und ein Start-Aufsetzpunkt auf eine Seite der Straße gesetzt, während der Ziel-Aufsetzpunkt auf der anderen Seite steht. Über das Auswahlfeld ID kann dann noch festgelegt werden, welcher Fußgänger über die Straße gehen soll (siehe Referenz, Kapitel 4.1). Die notwendigen Einstellungen sind in der folgenden Grafik zusammengefasst.

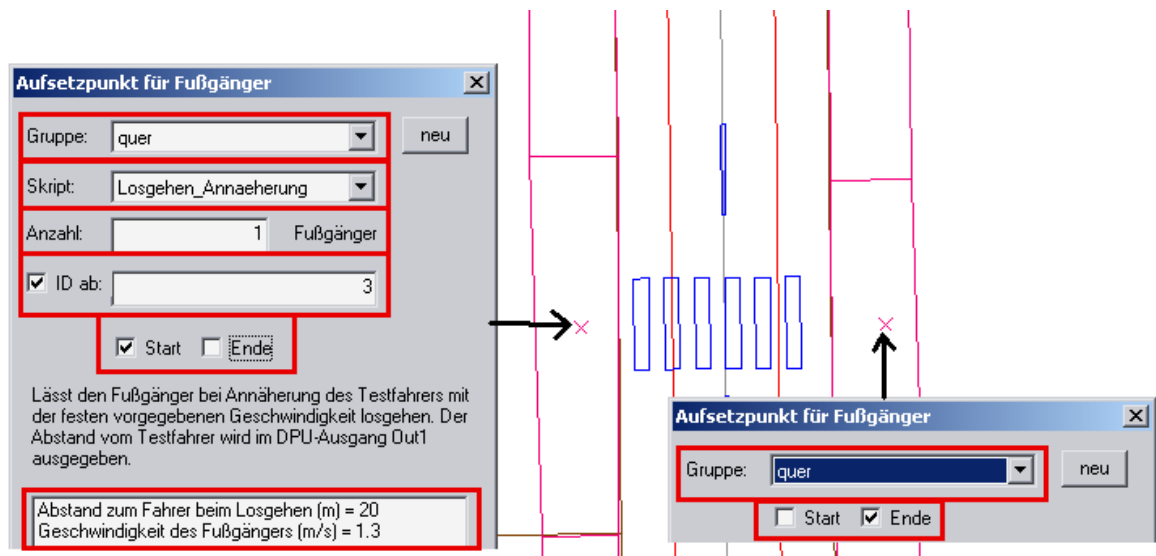


Abbildung 4 Beispiel für den Einsatz von Skripten

3.1.3 Wichtige Hinweise und Einschränkungen

Damit die Simulation der Fußgänger korrekt funktioniert, sollten folgende Punkte beachtet werden:

Fußgängerbereiche genau platzieren

Überall, wo getrennte Fußgängerbereiche aneinander stoßen, sollte darauf geachtet werden, dass diese genau aneinander passen. Überlappen sich die Bereiche um mehr als etwa $\frac{1}{2}$ m, oder ist mehr als $\frac{1}{2}$ m Platz zwischen ihnen, so werden sie von MOV als nicht zusammengehörig betrachtet. Die Fußgänger bleiben dann wie an einer Wand stehen.

Da durch das Einfügen eines Fußgängerüberwegs implizit ein Fußgängerbereich erzeugt wird, sollten Sie ebenfalls darauf achten, dass an diesen Stellen der Fußgängerbereich genau bis an den Rand der Straße geführt wird, wenn die Fußgänger den Überweg benutzen sollen.

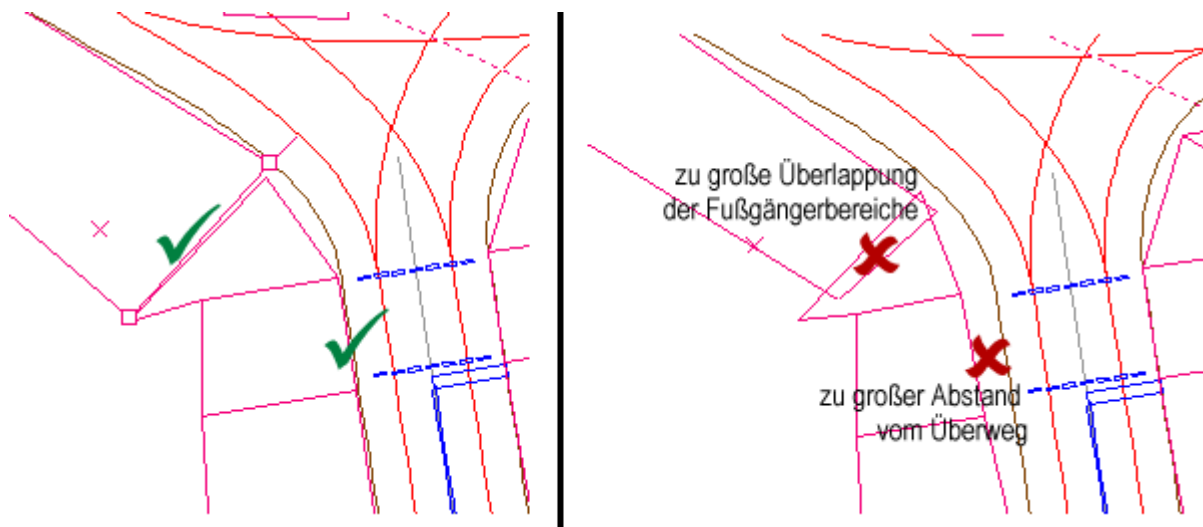


Abbildung 5 Anschluss von Fußgängerbereichen

Fußgängerbereiche nicht bis an Ein- und Ausfahrten der Area2 heranführen

Für die Simulation gilt folgende Einschränkung:

- Fußgänger können nur innerhalb von Area2's simuliert werden.
- Wenn die Standardeinstellungen der DPUMOV verwendet werden, werden zu jedem Zeitpunkt der Simulation nur die Fußgänger simuliert, die sich innerhalb derselben Area2-Instanz befinden wie der Testfahrer.
- Wenn zwei Verkehrsknoten (Area2) direkt aneinandergehängt werden, können auf jeden Fall nur die Fußgänger innerhalb des Verkehrsknotens simuliert werden, in dem sich der Testfahrer derzeit befindet.

Aus diesen Einschränkungen folgt, dass die Fußgänger im Normalfall bei der Einfahrt in einen Verkehrsknoten plötzlich erscheinen und bei der Ausfahrt ebenso schlagartig wieder verschwinden. Dies muss beim Entwurf des Verkehrsknotens berücksichtigt werden, d.h. die Fußgängerbereiche sollten nicht im Bereich der Sichtschutz-Kurven des Verkehrsknotens platziert werden (Sichtschutz-Kurven werden in den Schulungsunterlagen zu SILABAEEdit beschrieben).

Seit SILAB 4.0 ist es allerdings möglich, Fußgänger, die innerhalb einer nahegelegenen Area2-Instanz definiert sind, zu aktivieren, während sich der Testfahrer in einem Course befindet. Dazu werden die Parameter *PreActivateDistance* und *LeaveActiveNodes* der DPUMOV verwendet.

Diese Optionen ermöglichen es, einsehbare Kreuzungen (insbesondere in Überland-Szenarien) zu realisieren, in denen Fußgänger simuliert werden.

Aufsetzpunkte nur innerhalb von Fußgängerbereichen platzieren

Auch wenn es vielleicht trivial klingt, sollten Sie stets überprüfen, dass alle Aufsetzpunkte innerhalb von Fußgängerbereichen liegen. Gerade, wenn man Bereiche oder Straßenquerschnitte verschoben oder gelöscht hat, kann es leicht passieren, dass ein Aufsetzpunkt aus den Fußgängerbereichen herausrutscht. Dies führt dazu, dass keine Fußgänger mehr simuliert und angezeigt werden.

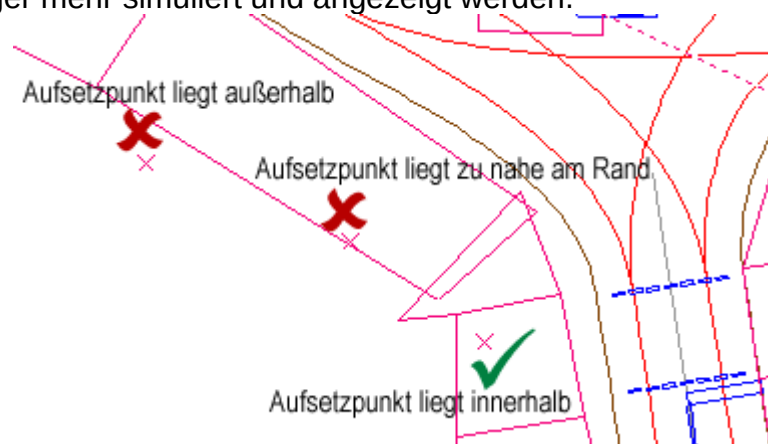


Abbildung 6 Lage der Aufsetzpunkte für Fußgänger

Mindestens zwei Aufsetzpunkte pro Fußgängergruppe werden benötigt

Beim Einsetzen eines Fußgängers in die Simulation wählt MOV stets einen Start- und einen davon verschiedenen Zielpunkt aus den Aufsetzpunkten aus, die der Fußgängergruppe zugeordnet sind, zu der der Fußgänger gehört. Daher müssen für jede

Fußgängergruppe (auch für die „ambient“-Gruppe, falls diese Fußgänger enthält) mindestens zwei Aufsetzpunkte definiert sein. Bei benutzerdefinierten Gruppen muss mindestens einer davon als Start- und ein anderer als Zielpunkt aktiviert sein.

3.2 Scripting der MOV-Detailkontrolle

Mit der MOV-Detailkontrolle können Fußgänger oder Fußgängergruppen in Simulationen eingefügt werden, die ein benutzerdefiniertes, versuchsspezifisches Verhalten besitzen.

Beispielsweise können so Fußgänger definiert werden, die bei Annäherung des Testfahrers die Straße überqueren (siehe Abschnitt „Spezial-Einsatz von Fußgängern“, Seite 7).

Zur Definition der Verhaltensmuster wird eine einfache Skriptsprache verwendet, die in Kap. 3.2.2 erläutert wird.

Die definierten Skripte können dann über eine „Verkehrsdefinition“ in SILABAEEdit eingebunden und in einer Area2 verwendet werden, dies wird in Kap. 3.2.3 erläutert.

3.2.1 Fußgängergruppen

Um die MOV-Detailkontrolle zu verstehen, ist es zunächst nötig, noch etwas mehr über Fußgängergruppen aus Sicht der Simulation zu erfahren.

Zur Steuerung der Fußgänger werden diese in Gruppen (*PedGroup*) unterteilt. Alle Fußgänger einer Gruppe werden gleich behandelt; auf sie wirken dieselben Ereignisse und sie halten sich auch im selben Bereich des Area2-Straßenabschnitts auf (dies wird durch die Wahl der möglichen Start- und Zielpunkte festgelegt).

Die meisten Eigenschaften einer Fußgängergruppe (z.B. Anzahl der Personen in der Gruppe, zugewiesene Start- und Zielpunkte) können direkt in SILABAEEdit über den Konfigurationsdialog eines Fußgänger-Setup-Punkts festgelegt werden.

Die Zuordnung eines Fußgängers zu einer Gruppe ist übrigens nicht fest, denn sie kann sich während der Simulation des Straßenabschnitts ändern (siehe Aktion *ChangeGroup*). Die Einstellungen in den Setup-Punkten beschreiben also lediglich, wie die zur Verfügung stehenden Fußgänger auf die Gruppen verteilt werden, wenn der Testfahrer in den Straßenabschnitt gelangt.

3.2.2 Skriptsprache

Skripte für die MOV-Detailkontrolle werden in einer einfachen Syntax verfasst, die im Folgenden erläutert ist.

Der Entwurf eines Skripts erfolgt dabei ereignisgesteuert: Für jedes Ereignis (*Event*) werden Aktionen (*Actions*) definiert, die ausgeführt werden sollen, immer wenn bestimmte Bedingungen (*Conditions*) erfüllt sind.

Zur Laufzeit der Simulation werden dann für jeden Fußgänger, der zu der Fußgängergruppe (*PedGroup*) gehört, für die das Skript eingesetzt wird, die definierten Ereignisse ausgeführt. Immer wenn der Fußgänger seinen nächsten Schritt plant (dies geschieht etwa zwei Mal pro Sekunde), werden die Bedingungen aller definierten Ereignisse abgeprüft. Die Aktionen aller aktivierten Ereignisse werden dann der Reihe nach ausgeführt.

Zusätzlich kann jedes Skript weitere Definitionen enthalten, die das Verhalten der Fußgänger beeinflussen.

Insgesamt sieht ein Skript wie folgt aus:

Skript für die MOV-Detailkontrolle

<Definitions>
(0..*)<Event>

3.2.2.1 Definitionen

Skript-Definitionen

<Definitions> →
(0..1)**PathPlanner** = <Zahl>;

Über die *Definitions* können bestimmte Eigenschaften der Fußgängergruppe festgelegt werden. Momentan ist hier nur die Angabe *PathPlanner* möglich (aber nicht vorgeschrieben).

PathPlanner

Mit dem Parameter *PathPlanner* kann für diese Fußgängergruppe ein anderer Pfadplaner, als für das komplette Area2 im Export-Dialog vorgegeben wurde, verwendet werden. Zur Auswahl stehen folgende Pfadplaner:

Zahl	Name	Funktion
0	Shortest Path	Geht von Start zu Ziel, wählt dann völlig neues Start und Ziel und teleportiert sich zum neuen Startpunkt.
1	Random Walk	Geht von Start zu Ziel, wählt dann ein neues Ziel und geht weiter zu diesem.
2	Explicit	Fußgänger ist standardmäßig inaktiv und geht nur dann los, wenn ihm dies über Skriptbefehle (<i>Replan</i> , <i>ReplanHere</i>) mitgeteilt wird. Ist er am Ziel angekommen, bleibt er stehen.

Beachten Sie, dass der Pfadplaner nur einmal beim Aktivieren des Area2s gewählt wird. Wenn die Fußgängergruppe über den Befehl *ChangeGroup* gewechselt wird, bleibt der Pfadplaner der alten Fußgängergruppe erhalten.

Für benutzerdefinierte Skripte besonders interessant ist der Explicit-Pfadplaner, weil damit aus dem Skript eine sehr genaue Kontrolle des Verhaltens der Fußgänger möglich ist. Die „normalen“ Pfadplaner (Shortest Path und Random Walk) dagegen führen viele Aktionen automatisch aus, z.B. die Suche eines neuen Wegs, wenn der Fußgänger seinem Ziel recht nahe kommt.

3.2.2.2 Ereignisse (Event)

Definition von Ereignissen

```

<Event> →
    Event <Name> {
        Conditions = {
            (0...*)<Condition>_(1)
        };
        Actions = {
            (1...*)<Action>_(1)
        };
    };

```

Mit Hilfe von Ereignissen (*Events*) ist es möglich, sehr detailliert die Reaktion von den Fußgängern der umgebenden *PedGroup* auf bestimmte Situationen festzulegen. Ein Ereignis besteht dabei aus einer Folge von Bedingungen, die alle gleichzeitig erfüllt sein müssen, damit es aktiviert wird (*Conditions*) und einer Folge von Aktionen, die bei Aktivierung der Reihe nach ausgeführt werden.

Für jeden Fußgänger, der sich derzeit in der jeweiligen *PedGroup* befindet, werden immer dann, wenn der nächste Geh-Schritt auszuführen ist (also etwa zwei Mal pro Sekunde) die *Conditions* aller definierten *Events* der *PedGroup* überprüft. Der Reihe nach werden dann alle *Actions* aller *Events*, die aktiviert werden, ausgeführt.

3.2.2.3 Bedingungen (Conditions)

Definition von Bedingungen

```

<Condition> →
    <Funktionsname>_(1...*)<Parameter>_(1)

```

Eine Bedingung ist eine Funktion, die entweder WAHR oder FALSCH sein kann. Es stehen zahlreiche Funktionen zur Auswahl, angefangen von arithmetischen Vergleichen von Variablen bis hin zu Abstandsabfragen (Bestimmung und Überprüfung des Abstands vom betrachteten Fußgänger zu gewissen Setup-Punkten, Fahrzeugen oder anderen Fußgängern). Beachten Sie, dass alle angegebenen *Conditions* automatisch UND-verknüpft werden, d.h. alle *Conditions* müssen gleichzeitig erfüllt sein, damit die Aktionen ausgelöst werden.

Die Anzahl und Art der Parameter hängt von der Art der Bedingung ab. Meist handelt es sich aber um beliebige Variablen (siehe dazu Kap. 3.2.2.5). Es folgt eine Aufzählung aller verfügbaren Bedingungen. Dabei kann für XX stets ein beliebiger der arithmetischen Vergleichstypen **Equal**, **NotEqual**, **Less**, **GreaterOrEqual**, **Greater**, **LessOrEqual** eingefügt werden.

XX(<Var1>,<Var2>)

Führt einen arithmetischen Vergleich der Variablen durch.

Beispiele: **Less(0.5, 1.2)** wird stets WAHR ergeben, während **Greater(p0, 1)** nur dann WAHR ergibt, wenn die Variable **p0** (erste private Variable des Fußgängers) größer als 1 ist.

CarDistanceXX(<VarDist>)

Prüft, ob die Entfernung von diesem Fußgänger zum Testfahrer xx (größer, kleiner, usw.) als der aktuelle Wert der Variablen <VarDist> ist.

CarDistanceXX(<VarDist>, <VarTrfID>)

Prüft, ob die Entfernung von diesem Fußgänger zum Fahrzeug mit Traffic-ID <VarTrfID> xx als <VarDist> ist.

CarDistanceXX(<VarDist>, <VarTrfID>, <VarSetupID>)

Prüft, ob der Abstand vom MOV-Setup-Punkt <VarSetupID> zum Fahrzeug mit Traffic-ID <VarTrfID> xx als <VarDist> ist.

CarSpeedXX(<VarValue>)

Prüft, ob die Geschwindigkeit des Testfahrers xx als <VarValue> ist.

CarSpeedXX(<VarValue>, <VarTrfID>)

Prüft, ob die Geschwindigkeit des Fahrzeugs mit Traffic-ID <VarTrfID> xx als <VarValue> ist.

DistanceXX(<VarDist>, <VarSetupID>)

Prüft, ob die Entfernung von diesem Fußgänger zum Setup-Punkt <VarSetupID> xx als <VarDist> ist.

PedDistanceXX(<VarDist>)

Prüft, ob der Abstand dieses Fußgängers vom nächsten anderen Fußgänger xx als <VarDist> ist.

PedDistanceXX(<VarDist>, <VarMovID>)

Prüft, ob der Abstand dieses Fußgängers zum Fußgänger mit der Moving-People-ID <VarMovID> xx als <VarDist> ist.

PedAngleXX(<VarDist>)

Prüft, ob der Winkel, in dem sich der nächste Fußgänger gegenüber unserer Bewegungsrichtung befindet, xx als <VarDist> ist.
Der Winkel wird in Grad angegeben. Beim Wert 0 wäre der Fußgänger exakt vor uns, positive Werte sind nach rechts, negative nach links.

PedAngleXX(<VarDist>, <VarMovID>)

Prüft, ob der Winkel, in dem sich der Fußgänger Moving-People-ID <VarMovID> gegenüber unserer Blickrichtung befindet, xx als <VarDist> ist.
Der Winkel wird in Grad angegeben. Beim Wert 0 wäre der Fußgänger exakt vor uns, positive Werte sind nach rechts, negative nach links.

IsBlockedXX(<VarSetupID>, <VarValue>)

Prüft, ob der Fußgängerbereich unter dem Setup-Punkt <VarSetupID> xx blockiert als <VarValue> ist. Fußgängerüberwege, an denen die Fußgängerampel rot ist, und Haltestellenbereiche, an denen kein Fahrzeug steht, haben den Blockierungswert 1.0. Freie Fußgängerbereiche haben den Blockierungswert 0.0.

3.2.2.4 Aktionen (Actions)

Definition von Aktionen

$\langle \text{Action} \rangle \rightarrow$
 $\langle \text{Funktionsname} \rangle ((1..*) \langle \text{Parameter} \rangle_{(i)})$

Eine Aktion ändert den Zustand der Fußgängersimulation. Es sind zahlreiche unterschiedliche Aktionen verfügbar, angefangen vom Setzen einer Variable auf einen Wert, über diverse Rechenfunktionen, bis hin zu einer Neuplanung des Wegs oder der Verschiebung des Fußgängers in eine andere *PedGroup*.

Die Anzahl und Art der Parameter einer Aktion hängt von deren Typ ab. Bei allen Aktionen, die ein Ergebnis erzeugen (z.B. Rechnungen) ist aber der erste Parameter eine Zielvariable, in die das Ergebnis abgelegt wird, und muss beschreibbar sein. Alle weiteren Parameter sind Quellvariablen und müssen nur gelesen werden können.

Es folgt eine Aufzählung aller verfügbaren Aktionen:

Set($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Setzt $\langle \text{VarZiel} \rangle$ auf den aktuellen Wert von $\langle \text{VarA} \rangle$.

Sin($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Cos($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Tan($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Setzt $\langle \text{VarZiel} \rangle$ auf den Wert der trigonometrischen Funktion (sin, cos, bzw. tan) für den Winkel $\langle \text{VarA} \rangle$. Dieser ist in Grad anzugeben.

Beispiel: **Sin**(**p0**, **30**) setzt die Variable **p0** auf den Sinus von 30°, also ½.

Exp($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Setzt $\langle \text{VarZiel} \rangle$ auf den Wert $\exp(\langle \text{VarA} \rangle)$, also $e^{\langle \text{VarA} \rangle}$ mit der eulerschen Zahl $e=2,718\dots$

Log($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$)

Setzt $\langle \text{VarZiel} \rangle$ auf den Wert des natürlichen Logarithmus von $\langle \text{VarA} \rangle$.

Add($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$, $\langle \text{VarB} \rangle$)

Subtract($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$, $\langle \text{VarB} \rangle$)

Multiply($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$, $\langle \text{VarB} \rangle$)

Divide($\langle \text{VarZiel} \rangle$, $\langle \text{VarA} \rangle$, $\langle \text{VarB} \rangle$)

Setzt $\langle \text{VarZiel} \rangle$ auf den Wert der arithmetischen Berechnung von $\langle \text{VarA} \rangle$ und $\langle \text{VarB} \rangle$.

Beispiele: **Add**(**p0**, **3**, **5**) setzt **p0** auf 8. **Divide**(**o3**, **2.2**, **1.1**) setzt **o3** auf 2.

CarDistance($\langle \text{VarZiel} \rangle$)

Ermittelt die Entfernung von diesem Fußgänger zum Testfahrer und speichert sie in der Variablen $\langle \text{VarZiel} \rangle$ ab.

CarDistance($\langle \text{VarZiel} \rangle$, $\langle \text{VarTrfID} \rangle$)

Ermittelt die Entfernung von diesem Fußgänger zum Fahrzeug mit der Traffic-ID $\langle \text{VarTrfID} \rangle$ und speichert sie in der Variablen $\langle \text{VarZiel} \rangle$ ab.

CarDistance($\langle \text{VarZiel} \rangle$, $\langle \text{VarTrfID} \rangle$, $\langle \text{VarSetupID} \rangle$)

Ermittelt die Entfernung vom Fahrzeug mit der Traffic-ID <VarTrfID> zum MOV-Setup-Punkt <VarSetupID> und speichert sie in der Variablen <VarZiel> ab.

CarSpeed(<VarZiel>)

Ermittelt die aktuelle Geschwindigkeit des Testfahrers und speichert sie in der Variablen <VarZiel> ab.

CarSpeed(<VarZiel>, <VarTrfID>)

Ermittelt die aktuelle Geschwindigkeit des Fahrzeugs mit der Traffic-ID <VarTrfID> und speichert sie in der Variablen <VarZiel> ab.

Distance(<VarZiel>, <VarSetupID>)

Ermittelt den Abstand dieses Fußgängers zum Setup-Punkt mit der ID <VarSetupID> und legt ihn in der Variablen <VarZiel> ab.

PedDistance(<VarZiel>)

Bestimmt den Abstand zum Fußgänger, der diesem am nächsten ist, und speichert ihn in der Variablen <VarZiel> ab.

PedDistance(<VarZiel>, <VarMovID>)

Bestimmt den Abstand zum Fußgänger mit der Moving-People-ID <VarMovID> und speichert ihn in der Variablen <VarZiel> ab.

PedAngle(<VarZiel>)

Bestimmt den Winkel, in dem sich der nächste Fußgänger gegenüber unserer Bewegungsrichtung befindet, und speichert ihn in <VarZiel> ab. Der Winkel wird in Grad angegeben. Beim Wert 0 wäre der Fußgänger exakt vor uns, positive Werte sind nach rechts, negative nach links.

PedAngle(<VarZiel>, <VarMovID>)

Bestimmt den Winkel, in dem sich der Fußgänger Moving-People-ID <VarMovID> gegenüber unserer Blickrichtung befindet, und speichert ihn in <VarZiel> ab.

Der Winkel wird in Grad angegeben. Beim Wert 0 wäre der Fußgänger exakt vor uns, positive Werte sind nach rechts, negative nach links.

ChangeGroup(<GroupName>)

Verschiebt den Fußgänger, für den diese Aktion ausgeführt wird, in die *PedGroup* mit Namen <GroupName>. Für <GroupName> kann auch **argx** angegeben werden. Dann wird Argument Nr. x (ab 0 gezählt) als Name der Ziel-*PedGroup* verwendet.

Achtung: Um Probleme zu vermeiden, sollte diese Aktion stets als letzte in einer *Actions*-Liste stehen. Diese Aktion ändert **nicht** den aktiven Pfadplaner, selbst wenn die neue Gruppe standardmäßig einen anderen Pfadplaner benutzt.

Replan(<VarSetupID>,<VarGoalID>)

Führt sofort eine neue Wegplanung aus. Der Weg des aktiven Fußgängers wird nun vom Setup-Punkt mit ID <VarSetupID> aus zum Setup-Punkt mit ID <VarGoalID> führen. Der Fußgänger wird direkt zum Startpunkt des neuen Weges befördert.

Beispiel: Nach der Ausführung von **Replan(1,4)** wird der Fußgänger zum Setup-Punkt 1 teleportiert und dann von dort aus zum Setup-Punkt 4 weitergehen.

ReplanHere(_(1..*)<VarSetupID>_(i))

Führt sofort eine neue Wegplanung aus. Der Weg des aktiven Fußgängers wird nun von seiner aktuellen Position zu einem Setup-Punkt, der zufällig aus allen angegebenen <VarSetupID>s ausgewählt wird, gehen.

Beispiel: Nach der Ausführung von **ReplanHere(4)** wird der Fußgänger von seiner aktuellen Position aus direkt zum Setup-Punkt 4 weitergehen.

MakeSpeed(<VarZiel>, <VarDesiredDist>, <VarDesiredTime>)

Bestimmt, auf welche Geschwindigkeit **sBaseSpeed** gesetzt werden müsste, damit dieser Fußgänger – ausgehend von seinem aktuellen Zustand – in <VarDesiredTime> Sekunden die Wegstrecke <VarDesiredDist> zurücklegt. Dabei wird berücksichtigt, dass der Fußgänger erst einmal auf die berechnete Geschwindigkeit beschleunigen bzw. abbremsen muss.

Achtung: Die Berechnung ist ziemlich aufwändig und sollte nicht unnötig oft durchgeführt werden.

Achtung: Falls die vorgegebene Distanz nicht in der vorgegebenen Zeit zurückgelegt werden kann (da sie zu groß ist) wird die Maximalgeschwindigkeit des Fußgängers zurückgegeben.

Beispiel: Nach der Ausführung von **MakeSpeed(sBaseSpeed, 4, 4)** wird der Fußgänger so beschleunigen bzw. abbremsen, dass er nach 4 Sekunden 4 Meter ($\pm 0,1$ m) zurückgelegt hat. (Egal ob der Fußgänger derzeit steht oder schon schnell geht, ist dieses Beispiel, zumindest für erwachsene Fußgänger, stets möglich.)

SetActiveForce(_(1..*)<Kraft>_(i))

Legt fest, welche Kräfte zur Berechnung des Fußgängerverhaltens berücksichtigt werden. Folgende Kräfte stehen zur Auswahl, es können beliebig viele davon angegeben werden:

Goal Zieht den Fußgänger in Richtung seines Ziels, das durch den Pfadplaner bestimmt wurde.

Wall Hält den Fußgänger von Wänden und anderen statischen Hindernissen fern und verhindert das Verlassen der Fußgängerbereiche.

People Hält den Fußgänger von anderen Personen fern.

Vehicle Hält den Fußgänger von Fahrzeugen (Testfahrer oder simulierter Verkehr) fern.

Crossing Verhindert das Betreten gesperrter Fußgängerüberwege.

Extra Zusätzliche Kraft zur freien Verwendung (s. **SetExtraForce()**).

Beim Start der Simulation werden alle Kräfte bis auf **Extra** aktiviert. Dieser Zustand kann durch den Aufruf **SetActiveForce(Default)** wiederhergestellt werden.

SetExtraForceXY(<VarX>, <VarY>)

SetExtraForceFS(<VarF>, <VarS>)

Setzt die zusätzliche anwendungsdefinierte Kraft, die benutzt werden kann, um gezielt Einfluss auf das Verhalten des Fußgängers zu nehmen.

Achtung: Beim Simulationsstart ist die anwendungsdefinierte Kraft deaktiviert. Sie muss zunächst durch **SetActiveForce()** aktiviert werden.

Die Kraft kann entweder in absoluten Weltkoordinaten <VarX> und <VarY> angegeben werden, oder sie kann im lokalen Koordinatensystem des Fußgängers über die Front-Koordinate <VarF> (positiv nach vorn, negativ nach hinten) und die Seitwärts-Koordinate <VarS> (positiv nach rechts, negativ nach links) angegeben werden.

SetState(<Zustand>)

Ändert den Bewegungszustand des Fußgängers. Folgende Zustände sind möglich:

Walk Normale Bewegung gemäß dem Krätemodell

Wait Wartet an der aktuellen Position, wobei Wartebewegungen ausgeführt werden.

Stop Wartet an der aktuellen Position, wobei der Fußgänger still steht.

Sequence Führt eine Abfolge von Bewegungen durch, die durch Aufrufe von **SeqAdd()** definiert wird, wobei der Fußgänger geradeaus geht.

PathSequence Führt eine Abfolge von Bewegungen durch, die durch Aufrufe von **SeqAdd()** definiert wird, wobei der Fußgänger in Richtung seines Zielpunktes weitergeht.

SeqAdd((1..*)<AnimID>(<,>))

Fügt Animationsabschnitte zur Bewegungssequenz hinzu, die im Bewegungszustand Sequence abgespielt wird. **Hinweis:** **SeqAdd()** sollte innerhalb derselben Aktionsliste, und zwar nach dem Aufruf von **SetState([Path]Sequence)**, aufgerufen werden.

Die Parameter sind Zahlen, die die Animationsabschnitte eindeutig identifizieren. Sie können in den Konfigurationsdateien

SILAB\lib\configurations\MOV\MOP*.inc nachgeschlagen werden.

Als Spezialfall kann der Buchstabe **P** angegeben werden, er steht für den jeweils bevorzugten Nachfolge-Animationsabschnitt (*PreferredNext*) des zuletzt abgespielten.

SetSecondary(<Zustand>)

Ändert sekundäre Aktionen des Fußgängers. Folgende Zustände sind möglich:

Default Sekundäre Aktionen werden nur aus der Sekundär-Aktionssequenz (siehe **SecondaryAdd()**) abgespielt.

LookDown Solange dieser Zustand aktiv ist, schaut der Fußgänger nach unten und benutzt ein Mobiltelefon.

SecondaryAdd_(1..*)<AnimID>_(i)

Fügt Animationsabschnitte zur Sequenz von Sekundäraktionen hinzu, die vom Fußgänger ausgeführt werden, wenn dieser sich im **Default**-Sekundärzustand befindet.

Die Parameter sind Zahlen, die die Animationsabschnitte eindeutig identifizieren. Sie können in den Konfigurationsdateien SILAB\lib\configurations\MOV\MOP*.inc nachgeschlagen werden.

Als Spezialfall kann der Buchstabe **P** angegeben werden, er steht für den jeweils bevorzugten Nachfolge-Animationsabschnitt (*PreferredNext*) des zuletzt abgespielten.

3.2.2.5 Variablen

Die meisten Parameter von Bedingungen oder Aktionen können beliebige Variablen sein. Alle diese Variablen haben als Wert eine Gleitkomma-Zahl in doppelter Genauigkeit. Eine Vielzahl unterschiedlicher Variablen ist verfügbar, die für Berechnungen benutzt werden können:

Verfügbare Variablen

<Variable> →
 <Konstante> |
 <Spezial-Konstante> |
 <Private Variable> |
 <PedGroup-Variable> |
 <Eingabe-Variable> |
 <Ausgabe-Variable> |
 <Spezial-Variable>

Viele dieser Variablen (aber nicht alle) können auch als Ziel einer Berechnung verwendet werden.

Konstanten

<Konstante> → (*Dezimalzahl mit Punkt als Trenner*)

Konstanten sind feste Zahlenwerte, die direkt in der Konfigurationsdatei angegeben werden. Sie können **nicht als Ziel** einer Berechnung verwendet werden.

Beispiele:

0.5
 -37208
 3.49e-3

Spezial-Konstanten

<Spezial-Konstante> →
 arg[0-9] |
 start[0-9] |
 goal[0-9]

Als spezielle Konstanten stehen die Argumente der Fußgängergruppe zur Verfügung, die der Reihe nach mit **arg0**, **arg1**, usw. angesprochen werden. Diese werden

stets als numerische Werte interpretiert. Außerdem können die Start- und Zielpunkte eingesetzt werden. Diese werden der Reihe nach mit **start0**, **start1**, ... bzw. **goal0**, **goal1**, ... angesprochen. Es können nur so viele Konstanten angesprochen werden, wie auch angegeben wurden, Zugriffe auf höhere Indizes führen zu einer Fehlermeldung beim Einlesen der Konfiguration.

Die Spezial-Konstanten können **nicht als Ziel** einer Berechnung verwendet werden.

Private Variablen

<Private Variable> → **p[0-9]**

Für jeden Fußgänger stehen zehn private Variablen zur Verfügung, die mit **p0** bis **p9** bezeichnet sind. Diese beziehen sich stets auf den aktiven Fußgänger, für den die Aktion ausgeführt wird. Ein Zugriff auf die privaten Variablen der anderen Fußgänger ist nicht möglich.

Die Variablen werden beim Start jedes Streckenstücks auf 0 initialisiert.

PedGroup-Variablen

<PedGroup-Variable> → **g[0-9]**

Für jede *PedGroup* stehen weitere 10 Variablen zur Verfügung, die mit **g0** bis **g9** bezeichnet sind. Diese haben für alle Fußgänger der *PedGroup* denselben Wert. Die Variablen beziehen sich stets auf die Gruppe, der der aktive Fußgänger angehört. Ein Zugriff auf Gruppenvariablen der anderen *PedGroups* ist nicht möglich.

Die Variablen werden beim Start jedes Streckenstücks auf 0 initialisiert.

Eingabe-Variablen

<Eingabe-Variable> → **i[0-9]**

Es stehen zehn globale Eingabe-Variablen zur Verfügung, die mit **i0** bis **i9** bezeichnet sind. Diese entsprechen jederzeit den aktuellen Werten der DPU-Eingänge **In1** bis **In10** der DPUMOV. Dabei entspricht Variable **i0** dem Eingang **In1** usw. Diese Variablen sind nicht beschreibbar, d.h. sie können **nicht als Ziel** einer Berechnung verwendet werden.

Achtung: Beachten Sie die Index-Verschiebung zwischen DPU-Eingang und Variablenname.

Achtung: Bedingungen und Aktionen werden nicht in jedem Simulationstakt ausgeführt, sondern nur etwa zwei Mal pro Sekunde (nämlich immer dann, wenn ein neuer Geh-Schritt ausgeführt werden soll). Deswegen ist es z.B. nicht ratsam, ein Switch an den Eingang **In1** anzuschließen und es dann mit der Bedingung **Equal(i0, 1)** abzufragen: Denn das Switch setzt **In1** nur für einen Simulationstakt auf 1, dies wird also wahrscheinlich übersehen werden.

Ausgabe-Variablen

<Ausgabe-Variable> → **o[0-9]**

Es stehen zehn globale Ausgabe-Variablen zur Verfügung, die mit **o0** bis **o9** bezeichnet sind. Die DPU-Ausgänge der DPUMOV **Out1** bis **Out10** entsprechen stets den Werten von **o0** bis **o9**. Dabei entspricht Variable **o0** dem Eingang **Out1** usw. Diese Variablen können ausgelesen und beschrieben werden.

Achtung: Beachten Sie die Index-Verschiebung zwischen DPU-Ausgang und Variablenname.

Spezial-Variablen

Zusätzlich zu den bisher vorgestellten Variablen stehen einige wenige Spezial-Variablen zur Verfügung.

sBaseSpeed

Grundgeschwindigkeit dieses Fußgängers (in m/s). Sie wird anfangs zufällig und normalverteilt gewählt und kann durch Zugriff auf diese Variable ausgelesen oder auch geändert werden. Bei einer Zuweisung wird automatisch auf den gültigen Wertebereich (0 bis zur Maximalgeschwindigkeit dieses Fußgängers) beschränkt.

Der Fußgänger nimmt seine Grundgeschwindigkeit an, wenn er auf einer großen freien Fläche ohne Einfluss durch Hindernisse, Fahrzeuge oder andere Fußgänger unterwegs ist.

sCurSpeed

Aktuelle Geschwindigkeit dieses Fußgängers (in m/s). Diese Variable kann **nur gelesen** werden.

sHandedness

Händigkeit des Fußgängers. Bei Werten > 0 wird die rechte Hand zum Halten von Objekten benutzt, bei Werten < 0 die linke Hand. Standardmäßig wird die Variable zufällig und gleichverteilt initialisiert.

sSimTime

Zeit (in Sekunden) seit dem Start der Simulation. Diese Variable kann **nur gelesen** werden.

sModuleTime

Zeit (in Sekunden) seit dem Eintritt des Testfahrers in das aktive Streckenstück (Area2). Diese Variable kann **nur gelesen** werden.

sPrivilegeLevel

Aktuelle Stufe der Sonderrechte des Testfahrers (0-3). Entspricht der SILAB-Variablen `Buttons.Privilege`. Diese Variable kann **nur gelesen** werden.

sSeqPos

Anzahl von bereits abgearbeiteten Animationsabschnitten innerhalb der Bewegungssequenz, die im **[Path]Sequence**-Bewegungszustand abgespielt wird. Diese Variable kann **nur gelesen** werden.

sSeqLength

Anzahl von noch geplanten Animationsabschnitten innerhalb der Bewegungssequenz, die im **[Path]Sequence**-Bewegungszustand abgespielt wird. Wenn die Sequenz fertig abgespielt wurde, wird der Wert auf 0 sinken. Diese Variable kann **nur gelesen** werden.

sCurAreaType

Typ des Fußgängerbereichs, auf dem sich der Fußgänger aktuell bewegt. Diese Variable kann **nur gelesen** werden. Mögliche Werte sind:

- 0 Gehweg
- 1 Ampelgeregelter Überweg
- 2 Zebrastreifen oder ungeregelter Überweg
- 3 Fahrzeug an Haltestelle

sNextAreaType

Typ des Fußgängerbereichs, den der Fußgänger demnächst erreicht. Falls der Fußgänger innerhalb von 2 Metern keinen neuen Fußgängerbereich erreichen wird, wird der Typ des aktuellen Bereichs zurückgegeben.

Diese Variable kann **nur gelesen** werden.

Mögliche Werte: siehe **sCurAreaType**.

sCurAreaBlocked

Blockierungszustand des Fußgängerbereichs, auf dem sich der Fußgänger aktuell bewegt. Fußgängerüberwege, an denen die Fußgängerampel rot ist, und Haltestellenbereiche, an denen kein Fahrzeug steht, haben den Blockierungswert 1.0. Freie Fußgängerbereiche haben den Blockierungswert 0.0.

Diese Variable kann **nur gelesen** werden.

sNextAreaBlocked

Blockierungszustand des Fußgängerbereichs, den der Fußgänger demnächst erreicht. Falls der Fußgänger innerhalb von 2 Metern keinen neuen Fußgängerbereich erreichen wird, wird der Zustand des aktuellen Bereichs zurückgegeben.

Diese Variable kann **nur gelesen** werden.

Mögliche Werte: siehe **sCurAreaBlocked**.

sRandomValue

Gibt einen pseudozufälligen Wert zwischen 0.0 und 1.0 zurück. Jeder Fußgänger hat einen individuellen Zufallszahlengenerator. Der Generator wird bei jedem Wechsel des Area2 zurückgesetzt.

Diese Variable kann **nur gelesen** werden.

3.2.2.6 Anmerkung zu IDs

Bei vielen *Conditions* und *Actions* müssen als Parameter IDs von Setup-Punkten, Fußgängern oder Verkehrsteilnehmern angegeben werden, auf die sie sich beziehen.

Im Folgenden wird genauer erklärt, wie sich die ID des gewünschten Objekts bestimmt.

Setup-Punkte (VarSetupID)

Alle Setup-Punkte einer Area2 werden beim Export durchnummeriert, die IDs werden der Reihe nach (ab 0) vergeben.

Fußgänger (VarMovID)

Alle Fußgänger werden beim Eintreten in eine Area2 von 0 an zählend durchnummeriert. Über die Option „ID ab...“ im Dialog eines Aufsetzpunkts einer Area2 kann für bestimmte Fußgängergruppen auch eine andere Nummerierung festgelegt werden.

Verkehrsteilnehmer (VarTrfID)

Die TRF-ID wird intern in der Simulation festgelegt und kann derzeit nicht vom SILAB-Anwender bestimmt oder beeinflusst werden.

3.2.3 Einbindung in SILABAEEdit

Damit ein Skript der MOV-Detailkontrolle auch in SILABAEEdit ausgewählt und eingesetzt werden kann, muss es durch eine „importierte Verkehrsdefinition“ dem Editor bekannt gemacht werden.

Eine solche Verkehrsdefinition (die als CFG-Datei abgelegt werden sollte) kann unter Anderem auch *PedScript*-Definitionen enthalten, mit denen Fußgängerskripte bekannt gemacht werden:

PedScript-Block in der Verkehrsdefinition

```
<PedScript> →
  PedScript <Name> {
    NArguments = <Zahl>;
    (0..*)Argumenti = "<Beschreibung>";
    (0..*)Valuei = <Zahl>;
    Description = "<Beschreibung>";
    Script = "<Dateiname>";
  };
```

Die Angaben im *PedScript*-Block bestimmen das Aussehen des Setup-Punkt-Dialogs, wenn das angegebene Skript ausgewählt ist.

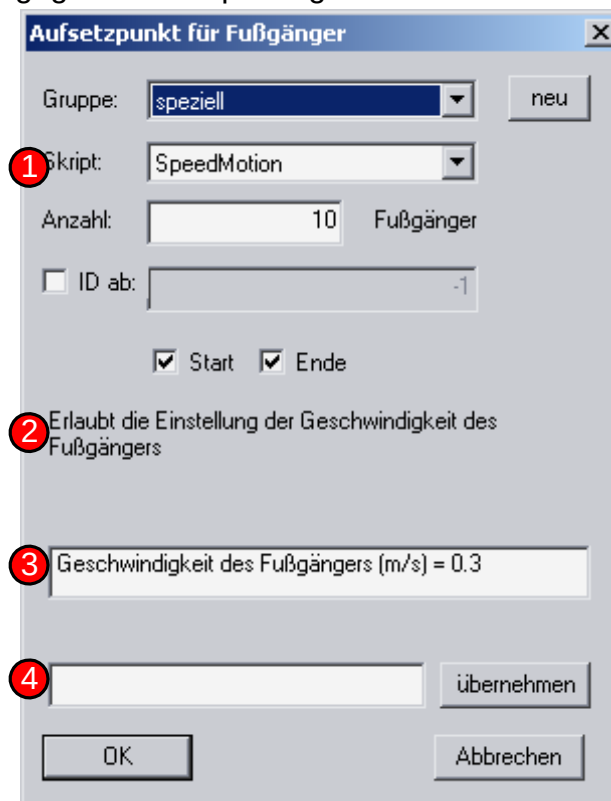


Abbildung 7 Elemente des Setup-Punkt-Dialogs in SILABAEEdit

Im Einzelnen bedeuten sie:

<Name>

Name des PedScripts, so wie er in der Auswahlbox **1** angezeigt wird. Darf nur aus Buchstaben und Zahlen bestehen und keine Leer- oder Sonderzeichen enthalten.

NArguments

Anzahl von einstellbaren Argumenten für das Skript. Die Argumente werden dem Skript in den Spezial-Konstanten **arg*i*** übergeben und können über das Listenfeld **3** für jede Fußgängergruppe separat festgelegt werden.

Argument*i*

Name des Arguments *i*, wie er im Listenfeld **3** angezeigt wird. Beachten Sie, dass die Indizes hier gegenüber dem Skript um 1 verschoben sind: Mit **Argument1** wird die Beschreibung für die Spezial-Konstante **arg0** festgelegt.

Value*i*

Standardwert für das Argument *i*. Dieser kann vom Benutzer über das Eingabefeld **4** geändert werden.

Description

Beschreibung des kompletten Fußgängerskripts. Diese wird im Textfeld **2** angezeigt.

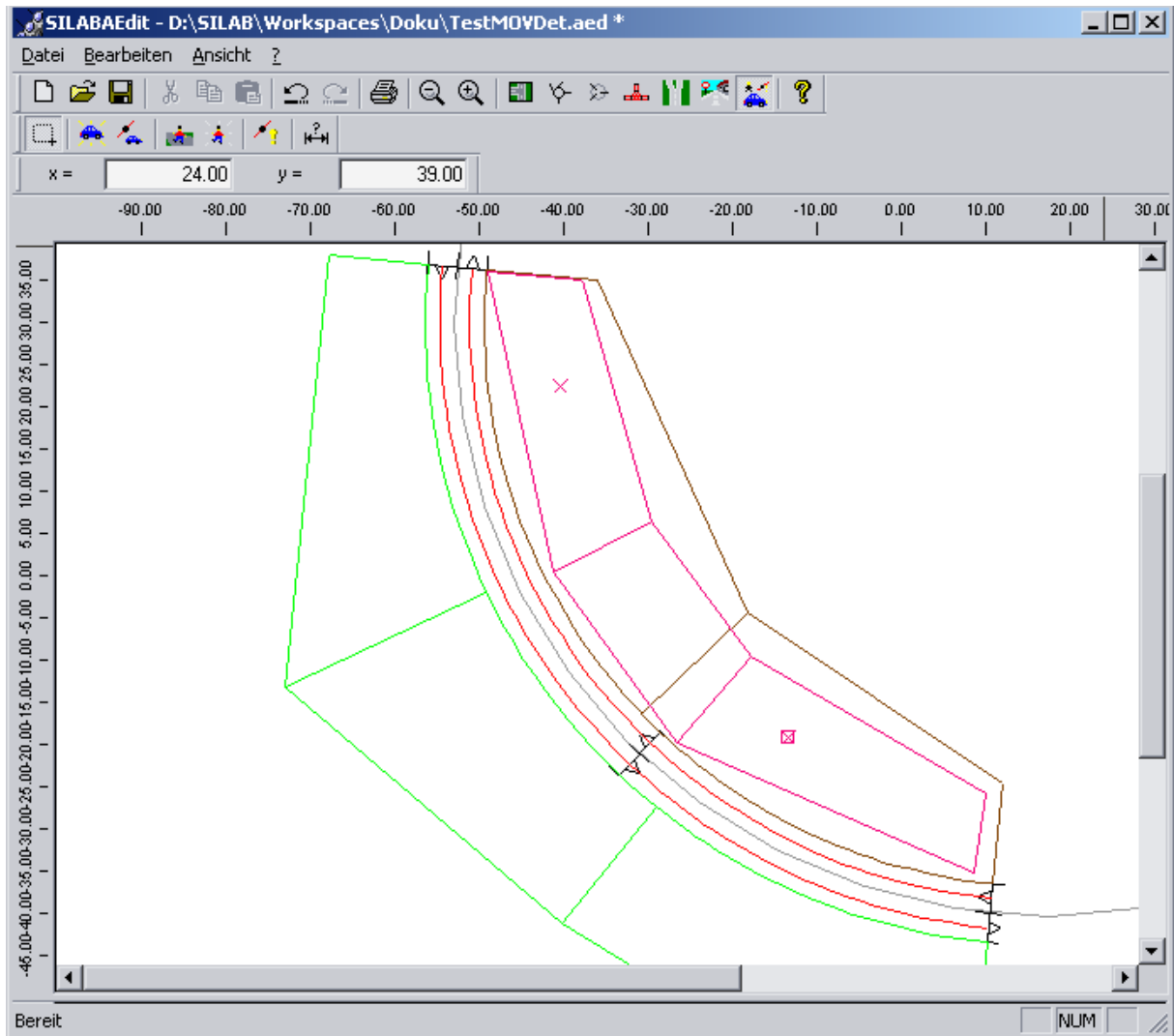
Script

Dateiname des Fußgängerskripts. Die Datei wird ausgehend von dem Verzeichnis gesucht, in dem sich die Verkehrsdefinitions-Datei befindet.

3.2.4 Beispiel

Als Beispiel soll ein benutzerdefiniertes Fußgängerskript erstellt werden, mit dem es möglich ist, den Fußgängern eine feste Wunschgeschwindigkeit vorzugeben.

- Erstellen Sie zunächst eine einfache Area2 mit SILABAEEdit. Definieren Sie einen Fußgängerbereich und zwei Aufsetzpunkte. Die Area2 könnte z.B. wie folgt aussehen:



- Speichern Sie die Area2 ab, zum Beispiel in das Verzeichnis „\SILAB\Projects\TestMovDet“.
- Exportieren Sie die Area2 mit folgenden Einstellungen:
 - Projekt-Pfad: \SILAB\Projects\TestMovDet
 - CFG-Datei: TestMovDet.cfg
 - Ankreuzen: Als Modul-Typ, mit TRF, mit MOP
 - Beliebige Namen für den Area2-Typ, Modul-Typ und Area2-Instanz vergeben
 - SIG-Datei: TestMovDet.sig
 - Testskript erzeugen (mit TRF und MOP), unter beliebigem Namen.
- Testen Sie die Area2 mit SILAB. Die Fußgänger werden sich mit normaler, zufällig verteilter Geschwindigkeit bewegen. (D.h. die Geschwindigkeit der einzelnen Fußgänger unterscheidet sich.)
- Legen Sie im Verzeichnis „\SILAB\Projects\TestMovDet“ eine Skriptdatei für die MOV-Detailkontrolle unter dem Namen „TestMovDet_SpeedMotion.inc“ an und geben Sie folgendes Skript ein:

```
Event Setup {
```



```

Conditions = {
    Equal(p0, 0) # Noch nicht ausgeführt
};
Actions = {
    Set(sBaseSpeed, arg0),
    Set(p0, 1)
};
};

```

Das Skript definiert ein einziges Ereignis, das beim Eintritt in das Area2 für jeden Fußgänger einmalig ausgeführt wird. (Dies wird durch die Bedingung „p0=0“ (Equal(p0, 0)) und die Aktion „p0 := 1“ (Set(p0, 1)) erreicht.) Dabei wird die Basisgeschwindigkeit des Fußgängers *sBaseSpeed* auf das erste Skriptargument *arg0* festgesetzt. (Achtung, im MOVDet-Skript fängt die Zählung stets bei 0 an!)

- Legen Sie im Verzeichnis „\SILAB\Projects\TestMovDet“ eine Verkehrsdefinition für SILABAEEdit als Datei „TestMovDet_trafficinfo.cfg“ mit folgendem Inhalt an:

```

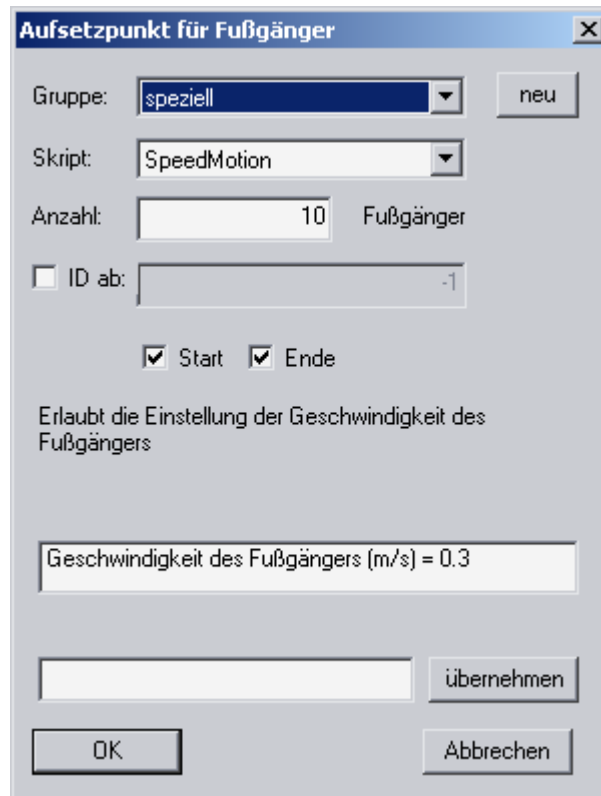
PedScript SpeedMotion
{
    NArguments = 1;
    Argument1 = "Geschwindigkeit des Fußgängers (m/s)";
    Value1 = 1.3;
    Description = "Erlaubt die Einstellung der Geschwindigkeit des Fußgängers";
    Script = "TestMOVDet_SpeedMotion.inc";
};

```

Hiermit wird das Fußgängerskript „SpeedMotion“ in SILABAEEdit eingebunden, so dass Sie es unter „Skript“ im Konfigurationsdialog für Setup-Punkte auswählen können.

Dabei wird festgelegt, dass das Skript ein Argument besitzt (*NArguments*=1). Die Beschreibung des ersten Arguments (Achtung, hier fängt die Zählung bei 1 an!) und der Standardwert werden festgelegt (*Argument1* und *Value1*). Zusätzlich wird noch eine Beschreibung des Skripts (*Description*) und ein Verweis auf den Dateinamen der Skriptdatei (*Script*) abgelegt.

- Legen Sie eine neue Fußgängergruppe (hier „speziell“, aber der Name ist beliebig) an und stellen Sie nun für beide Setup-Punkte der Area2 folgendes ein:



The screenshot shows a software dialog box titled "Aufsetzpunkt für Fußgänger". It contains several input fields and checkboxes. The "Gruppe:" dropdown is set to "speziell", and there is a "neu" button next to it. The "Skript:" dropdown is set to "SpeedMotion". The "Anzahl:" field is set to "10" with the label "Fußgänger" next to it. There is an unchecked checkbox for "ID ab:" followed by a field containing "-1". Below these are two checked checkboxes for "Start" and "Ende". A text label reads "Erlaubt die Einstellung der Geschwindigkeit des Fußgängers". A text field displays "Geschwindigkeit des Fußgängers (m/s) = 0.3". At the bottom, there is an empty text field, an "übernehmen" button, an "OK" button, and an "Abbrechen" button.

Aufsetzpunkt für Fußgänger

Gruppe:

Skript:

Anzahl: Fußgänger

☐ ID ab:

☒ Start ☒ Ende

Erlaubt die Einstellung der Geschwindigkeit des Fußgängers

- Nach einem erneuten Export der Area2 sollten sich alle Fußgänger nun nur noch im Schneckentempo bewegen.

4 REFERENZ

4.1 Referenz der Standard-Fußgänger

Die folgenden 35 Fußgängermodelle sind innerhalb der Gruppe „Normal“ vorhanden und werden standardmäßig genutzt, um Fußgängergruppen zu füllen.



ID=0
Größe: 1,84 m



ID=1
Größe: 1,82 m



ID=2
Größe: 1,77 m



ID=3
Größe: 1,65 m



ID=4
Größe: 1,69 m



ID=5
Größe: 1,73 m



ID=6
Größe: 1,70 m



ID=7
Größe: 1,88 m



ID=8
Größe: 1,83 m



ID=9
Größe: 1,35 m



ID=10
Größe: 1,23 m



ID=11
Größe: 1,89 m



ID=12
Größe: 1,61 m



ID=13
Größe: 1,79 m



ID=14
Größe: 1,70 m



ID=15
Größe: 1,76 m



ID=16
Größe: 1,63 m



ID=17
Größe: 1,68 m



ID=18
Größe: 1,74 m



ID=19
Größe: 1,82 m



ID=20
Größe: 1,80 m



ID=21
Größe: 1,35 m



ID=22
Größe: 1,23 m



ID=23
Größe: 1,72 m



ID=24
Größe: 1,80 m



ID=25
Größe: 1,80 m



ID=26
Größe: 1,71 m



ID=27
Größe: 1,74 m



ID=28
Größe: 1,79 m



ID=29
Größe: 1,66 m



ID=30
Größe: 1,75 m



ID=31
Größe: 1,69 m



ID=32
Größe: 1,88 m



ID=33
Größe: 1,77 m



ID=34
Größe: 1,74 m

4.2 Referenz der Sonder-Fußgänger

Die folgenden 14 Fußgängermodelle sind in der Gruppe „Spezial“ vorhanden und können auf Wunsch für Fußgängergruppen verwendet werden.



ID=0
Größe: 1,66 m



ID=1
Größe: 1,80 m



ID=2
Größe: 1,66 m



ID=3
Größe: 1,80 m



ID=4
Größe: 1,66 m



ID=5
Größe: 1,81 m



ID=6
Größe: 1,62 m



ID=7
Größe: 1,81 m



ID=8
Größe: 1,74 m



ID=9
Größe: 1,75 m



ID=10
Größe: 1,69 m



ID=11
Größe: 1,80 m



ID=12
Größe: 1,69 m



ID=13
Größe: 1,84 m

